

# Building DNNs and CNNs from Scratch — CIFAR-10 Classification

## Practice Assignment (AISV 802)

In this notebook we build an end-to-end image classification pipeline on the **CIFAR-10** dataset (10 classes, 60,000 32x32 color images) using `tf.data`, `tf.keras.Sequential`, and `tf.keras.Model` subclassing.

## Contents

1. Setup & imports
2. Load CIFAR-10 and build `tf.data` train/validation/test pipelines
3. Data augmentation inside the pipeline
4. DNN — Sequential API
5. DNN — Model subclassing API
6. Train & evaluate the DNN, learning curves
7. CNN — Sequential API
8. CNN — Model subclassing API
9. Train & evaluate the CNN, learning curves
10. TensorBoard visualization
11. Hyperparameter tuning (Keras Tuner)
12. Final comparison & conclusions

**Note:** This notebook is designed to run top-to-bottom on **Google Colab** (recommended, free GPU) or on a local machine with TensorFlow 2.x installed. `tf.keras.datasets.cifar10` downloads the dataset from the internet the first time it runs, so an internet connection is required.

## 1. Setup & Imports

```
# If running on Colab and a package is missing, uncomment the line below.
# !pip install -q keras-tuner

import os
import datetime
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print("TensorFlow version:", tf.__version__)
print("Keras version:", keras.__version__)
print("GPU available:", tf.config.list_physical_devices('GPU'))

# Reproducibility
SEED = 42
tf.random.set_seed(SEED)
np.random.seed(SEED)

# Clear any logs from previous runs
!rm -rf ./logs/
LOG_ROOT = "logs"
os.makedirs(LOG_ROOT, exist_ok=True)

TensorFlow version: 2.20.0
Keras version: 3.13.2
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

## 2. Load CIFAR-10 and Build `tf.data` Pipelines

CIFAR-10 ships with a train/test split (50,000 / 10,000 images). We carve out the last 10% of the training set as a **validation** set, so we end up with a proper train / validation / test pipeline as requested.

```
(x_train_full, y_train_full), (x_test, y_test) = keras.datasets.cifar10.load_data()

y_train_full = y_train_full.squeeze()
y_test = y_test.squeeze()

CLASS_NAMES = ['airplane', 'automobile', 'bird', 'cat', 'deer',
                'dog', 'frog', 'horse', 'ship', 'truck']
NUM_CLASSES = len(CLASS_NAMES)
INPUT_SHAPE = x_train_full.shape[1:] # (32, 32, 3)

# Hold out 10% of the training data for validation
val_frac = 0.1
n_val = int(len(x_train_full) * val_frac)

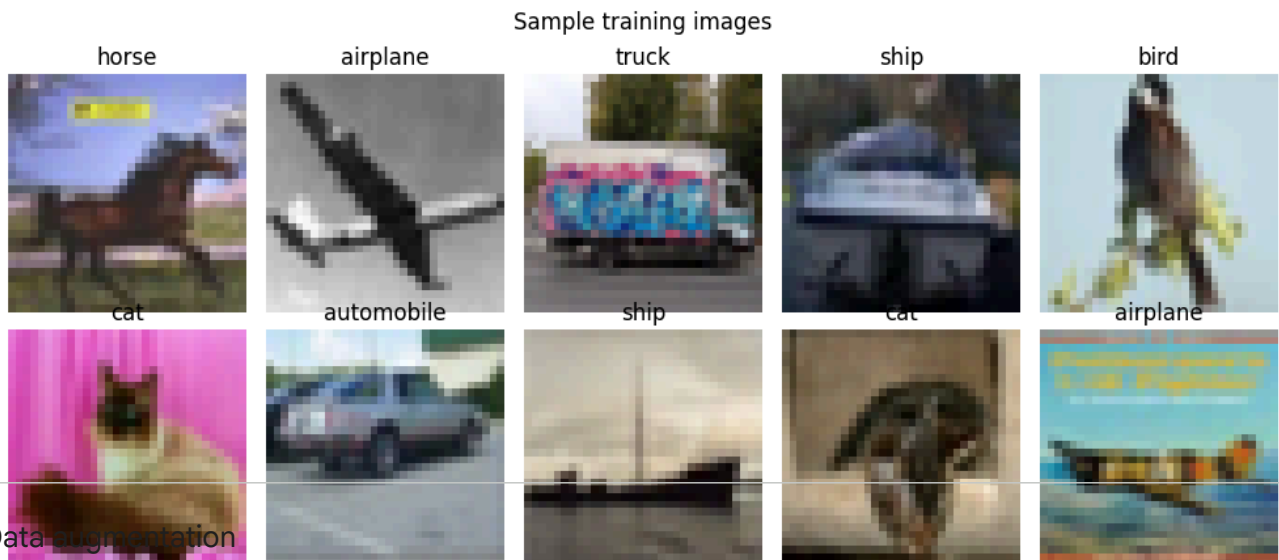
# Shuffle indices once so the validation split is not biased by class ordering
rng = np.random.default_rng(SEED)
perm = rng.permutation(len(x_train_full))
x_train_full, y_train_full = x_train_full[perm], y_train_full[perm]

x_val, y_val = x_train_full[:n_val], y_train_full[:n_val]
x_train, y_train = x_train_full[n_val:], y_train_full[n_val:]

print("Train:", x_train.shape, y_train.shape)
print("Val:  ", x_val.shape, y_val.shape)
print("Test: ", x_test.shape, y_test.shape)
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 — 1289s 8us/step
Train: (45000, 32, 32, 3) (45000,)
Val:   (5000, 32, 32, 3) (5000,)
Test:  (10000, 32, 32, 3) (10000,)
```

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4.5))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_train[i])
    ax.set_title(CLASS_NAMES[y_train[i]])
    ax.axis('off')
plt.suptitle('Sample training images')
plt.tight_layout()
plt.show()
```



### 2.1 Data augmentation

Augmentation is implemented as a small `tf.keras.Sequential` of preprocessing layers and is applied **inside the `tf.data` pipeline** (via `.map()`), only for the **training** split. This keeps the augmentation graph-compatible (runs on GPU) and keeps validation/test data untouched.

```
AUTOTUNE = tf.data.AUTOTUNE
BATCH_SIZE = 64

data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.06),
    layers.RandomZoom(0.1),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomContrast(0.1),
], name="data_augmentation")

normalization = layers.Rescaling(1. / 255)

def make_dataset(images, labels, training=False, batch_size=BATCH_SIZE, shuffle_buf=10000):
    ds = tf.data.Dataset.from_tensor_slices((images, labels))
    if training:
        ds = ds.shuffle(shuffle_buf, seed=SEED, reshuffle_each_iteration=True)
        ds = ds.map(lambda x, y: (tf.cast(x, tf.float32), y), num_parallel_calls=AUTOTUNE)
        if training:
            ds = ds.map(lambda x, y: (data_augmentation(x, training=True), y),
                          num_parallel_calls=AUTOTUNE)
        ds = ds.map(lambda x, y: (normalization(x), y), num_parallel_calls=AUTOTUNE)
        ds = ds.batch(batch_size)
        ds = ds.prefetch(AUTOTUNE)
    return ds

train_ds = make_dataset(x_train, y_train, training=True)
val_ds = make_dataset(x_val, y_val, training=False)
test_ds = make_dataset(x_test, y_test, training=False)

train_ds, val_ds, test_ds
```

```
(<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 32, 32, 3), dtype=tf.float32, name=None),
TensorSpec(shape=(None,), dtype=tf.uint8, name=None))>,
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 32, 32, 3), dtype=tf.float32, name=None),
TensorSpec(shape=(None,), dtype=tf.uint8, name=None))>,
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 32, 32, 3), dtype=tf.float32, name=None),
TensorSpec(shape=(None,), dtype=tf.uint8, name=None))>)
```

```
# Visualize a few augmented training images (same source image, multiple augmentations)
sample_img = tf.cast(x_train[0], tf.float32)

plt.figure(figsize=(10, 3))
for i in range(6):
    aug_img = data_augmentation(sample_img, training=True).numpy().astype("uint8")
    ax = plt.subplot(1, 6, i + 1)
    plt.imshow(aug_img)
    plt.axis('off')
plt.suptitle(f"Augmented versions of a single '{CLASS_NAMES[y_train[0]]}' image")
plt.tight_layout()
plt.show()
```

Augmented versions of a single 'horse' image



### 3. Deep Neural Network (DNN — fully connected)

We build a plain fully-connected network (no convolutions) as a baseline, using **both** the Sequential API and Model-subclassing API, as covered in class.

#### 3.1 DNN — `tf.keras.Sequential` API

```
def build_dnn_sequential(input_shape=INPUT_SHAPE, num_classes=NUM_CLASSES,
                        hidden_units=(512, 256, 128), dropout=0.3, l2=1e-4):
    reg = keras.regularizers.l2(l2)
    model = keras.Sequential(name="DNN_Sequential")
    model.add(keras.Input(shape=input_shape))
    model.add(layers.Flatten())
    for units in hidden_units:
        model.add(layers.Dense(units, kernel_regularizer=reg))
        model.add(layers.BatchNormalization())
        model.add(layers.Activation("relu"))
        model.add(layers.Dropout(dropout))
    model.add(layers.Dense(num_classes, activation="softmax"))
    return model
```

```
dnn_seq = build_dnn_sequential()
dnn_seq.summary()
```

Model: "DNN\_Sequential"

| Layer (type)                               | Output Shape | Param #   |
|--------------------------------------------|--------------|-----------|
| flatten (Flatten)                          | (None, 3072) | 0         |
| dense (Dense)                              | (None, 512)  | 1,573,376 |
| batch_normalization (BatchNormalization)   | (None, 512)  | 2,048     |
| activation (Activation)                    | (None, 512)  | 0         |
| dropout (Dropout)                          | (None, 512)  | 0         |
| dense_1 (Dense)                            | (None, 256)  | 131,328   |
| batch_normalization_1 (BatchNormalization) | (None, 256)  | 1,024     |
| activation_1 (Activation)                  | (None, 256)  | 0         |
| dropout_1 (Dropout)                        | (None, 256)  | 0         |
| dense_2 (Dense)                            | (None, 128)  | 32,896    |
| batch_normalization_2 (BatchNormalization) | (None, 128)  | 512       |
| activation_2 (Activation)                  | (None, 128)  | 0         |
| dropout_2 (Dropout)                        | (None, 128)  | 0         |
| dense_3 (Dense)                            | (None, 10)   | 1,290     |

Total params: 1,742,474 (6.65 MB)

Trainable params: 1,742,474 (6.65 MB)

#### 3.2 DNN — `tf.keras.Model` Subclassing API

```
class DNNSubclassed(keras.Model):
    """Fully-connected network built with the Model-subclassing API.
```

Functionally equivalent to `build\_dnn\_sequential`, but defined explicitly with `\_\_init\_\_` / `call`, giving full control over the forward pass.

"""

```
def __init__(self, num_classes=NUM_CLASSES, hidden_units=(512, 256, 128),
             dropout=0.3, l2=1e-4, **kwargs):
    super().__init__(name="DNN_Subclassed", **kwargs)
    reg = keras.regularizers.l2(l2)
    self.flatten = layers.Flatten()
    self.blocks = []
    for units in hidden_units:
        self.blocks.append([
            layers.Dense(units, kernel_regularizer=reg),
            layers.BatchNormalization(),
            layers.Activation("relu"),
            layers.Dropout(dropout),
        ])
    self.classifier = layers.Dense(num_classes, activation="softmax")

def call(self, inputs, training=False):
    x = self.flatten(inputs)
    for dense, bn, act, drop in self.blocks:
        x = dense(x)
        x = bn(x, training=training)
        x = act(x)
        x = drop(x, training=training)
    return self.classifier(x)
```

```
dnn_sub = DNNSubclassed()
dnn_sub.build(input_shape=(None,) + INPUT_SHAPE)
dnn_sub.summary()
```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/layer.py:424: UserWarning: `build()` was called on layer 'warnings.warn(

Model: "DNN\_Subclassed"

| Layer (type)                                                 | Output Shape | Param #     |
|--------------------------------------------------------------|--------------|-------------|
| flatten_1 ( <a href="#">Flatten</a> )                        | ?            | 0 (unbuilt) |
| dense_4 ( <a href="#">Dense</a> )                            | ?            | 0 (unbuilt) |
| batch_normalization_3 ( <a href="#">BatchNormalization</a> ) | ?            | 0 (unbuilt) |
| activation_3 ( <a href="#">Activation</a> )                  | ?            | 0           |
| dropout_3 ( <a href="#">Dropout</a> )                        | ?            | 0           |
| dense_5 ( <a href="#">Dense</a> )                            | ?            | 0 (unbuilt) |
| batch_normalization_4 ( <a href="#">BatchNormalization</a> ) | ?            | 0 (unbuilt) |
| activation_4 ( <a href="#">Activation</a> )                  | ?            | 0           |
| dropout_4 ( <a href="#">Dropout</a> )                        | ?            | 0           |
| dense_6 ( <a href="#">Dense</a> )                            | ?            | 0 (unbuilt) |
| batch_normalization_5 ( <a href="#">BatchNormalization</a> ) | ?            | 0 (unbuilt) |
| activation_5 ( <a href="#">Activation</a> )                  | ?            | 0           |
| dropout_5 ( <a href="#">Dropout</a> )                        | ?            | 0           |
| dense_7 ( <a href="#">Dense</a> )                            | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)  
 Trainable params: 0 (0.00 B)  
 Non-trainable params: 0 (0.00 B)

## 4. Train & Evaluate the DNN

We train the **Sequential** DNN as the primary experiment (with TensorBoard, early stopping and a learning-rate scheduler), then briefly fine-tune the **subclassd** DNN for a few epochs to confirm both APIs behave equivalently.

```
def make_callbacks(name, patience=6):
    log_dir = os.path.join(LOG_ROOT, name, datetime.datetime.now().strftime("%Y%m%d-%H%M%S"))
    return [
        keras.callbacks.TensorBoard(log_dir=log_dir, histogram_freq=1),
        keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=patience,
                                       restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3,
                                           min_lr=1e-6, verbose=1),
    ], log_dir
```

```
EPOCHS_DNN = 30
```

```
dnn_seq = build_dnn_sequential()
dnn_seq.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

```
cb_dnn, dnn_log_dir = make_callbacks("dnn_sequential")
history_dnn = dnn_seq.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS_DNN,
    callbacks=cb_dnn,
    verbose=2,
)
```

```
Epoch 1/30
704/704 - 36s - 52ms/step - accuracy: 0.3059 - loss: 2.0925 - val_accuracy: 0.3474 - val_loss: 1.9284 - learning_
Epoch 2/30
704/704 - 16s - 22ms/step - accuracy: 0.3697 - loss: 1.8954 - val_accuracy: 0.3034 - val_loss: 2.0935 - learning_
Epoch 3/30
704/704 - 16s - 22ms/step - accuracy: 0.3915 - loss: 1.8425 - val_accuracy: 0.2936 - val_loss: 2.1142 - learning_
Epoch 4/30
704/704 - 16s - 22ms/step - accuracy: 0.4003 - loss: 1.8122 - val_accuracy: 0.4060 - val_loss: 1.8421 - learning_
Epoch 5/30
704/704 - 16s - 22ms/step - accuracy: 0.4043 - loss: 1.8007 - val_accuracy: 0.3564 - val_loss: 1.9413 - learning_
Epoch 6/30
704/704 - 16s - 22ms/step - accuracy: 0.4084 - loss: 1.7915 - val_accuracy: 0.3714 - val_loss: 1.8815 - learning_
Epoch 7/30
```

```
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
```

```
704/704 - 16s - 22ms/step - accuracy: 0.4102 - loss: 1.7897 - val_accuracy: 0.3574 - val_loss: 1.9354 - learning_
Epoch 8/30
704/704 - 16s - 22ms/step - accuracy: 0.4306 - loss: 1.7261 - val_accuracy: 0.3998 - val_loss: 1.8028 - learning_
Epoch 9/30
704/704 - 16s - 22ms/step - accuracy: 0.4361 - loss: 1.7065 - val_accuracy: 0.3850 - val_loss: 1.8119 - learning_
Epoch 10/30
704/704 - 16s - 22ms/step - accuracy: 0.4366 - loss: 1.6949 - val_accuracy: 0.4224 - val_loss: 1.7320 - learning_
Epoch 11/30
704/704 - 16s - 23ms/step - accuracy: 0.4396 - loss: 1.6873 - val_accuracy: 0.4558 - val_loss: 1.6397 - learning_
Epoch 12/30
704/704 - 16s - 22ms/step - accuracy: 0.4462 - loss: 1.6730 - val_accuracy: 0.4196 - val_loss: 1.7371 - learning_
Epoch 13/30
704/704 - 16s - 22ms/step - accuracy: 0.4430 - loss: 1.6730 - val_accuracy: 0.3748 - val_loss: 1.8298 - learning_
Epoch 14/30
```

```
Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
```

```
704/704 - 16s - 22ms/step - accuracy: 0.4516 - loss: 1.6583 - val_accuracy: 0.4498 - val_loss: 1.6565 - learning_
Epoch 15/30
704/704 - 16s - 22ms/step - accuracy: 0.4596 - loss: 1.6270 - val_accuracy: 0.4900 - val_loss: 1.5424 - learning_
Epoch 16/30
```

```
704/704 - 16s - 23ms/step - accuracy: 0.4661 - loss: 1.6089 - val_accuracy: 0.4720 - val_loss: 1.5954 - learning_
Epoch 17/30
704/704 - 16s - 23ms/step - accuracy: 0.4667 - loss: 1.6002 - val_accuracy: 0.4766 - val_loss: 1.5617 - learning_
Epoch 18/30
```

Epoch 18: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

```
704/704 - 16s - 23ms/step - accuracy: 0.4677 - loss: 1.5935 - val_accuracy: 0.4642 - val_loss: 1.5975 - learning_
Epoch 19/30
704/704 - 16s - 22ms/step - accuracy: 0.4770 - loss: 1.5700 - val_accuracy: 0.5384 - val_loss: 1.4074 - learning_
Epoch 20/30
704/704 - 16s - 22ms/step - accuracy: 0.4789 - loss: 1.5585 - val_accuracy: 0.5328 - val_loss: 1.4064 - learning_
Epoch 21/30
704/704 - 16s - 22ms/step - accuracy: 0.4792 - loss: 1.5555 - val_accuracy: 0.5432 - val_loss: 1.3887 - learning_
Epoch 22/30
704/704 - 16s - 22ms/step - accuracy: 0.4823 - loss: 1.5473 - val_accuracy: 0.5314 - val_loss: 1.4074 - learning_
Epoch 23/30
704/704 - 16s - 23ms/step - accuracy: 0.4829 - loss: 1.5443 - val_accuracy: 0.5158 - val_loss: 1.4463 - learning_
Epoch 24/30
```

Epoch 24: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

```
704/704 - 16s - 22ms/step - accuracy: 0.4825 - loss: 1.5340 - val_accuracy: 0.5324 - val_loss: 1.4197 - learning_
Epoch 25/30
```

```
dnn_test_loss, dnn_test_acc = dnn_seq.evaluate(test_ds, verbose=0)
print(f"[DNN - Sequential] Test loss: {dnn_test_loss:.4f} | Test accuracy: {dnn_test_acc:.4f}")
```

```
[DNN - Sequential] Test loss: 1.3261 | Test accuracy: 0.5612
```

# Quick equivalence check: train the subclassed DNN for a handful of epochs

```
dnn_sub = DNNSubclassed()
dnn_sub.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
cb_dnn_sub, _ = make_callbacks("dnn_subclassed")
history_dnn_sub = dnn_sub.fit(
    train_ds, validation_data=val_ds, epochs=5, callbacks=cb_dnn_sub, verbose=2,
)
sub_test_loss, sub_test_acc = dnn_sub.evaluate(test_ds, verbose=0)
print(f"[DNN - Subclassed, 5 epochs] Test loss: {sub_test_loss:.4f} | Test accuracy: {sub_test_acc:.4f}")
```

```
Epoch 1/5
704/704 - 28s - 40ms/step - accuracy: 0.3067 - loss: 2.0903 - val_accuracy: 0.2806 - val_loss: 2.0832 - learning_ra
Epoch 2/5
704/704 - 16s - 23ms/step - accuracy: 0.3721 - loss: 1.8995 - val_accuracy: 0.3366 - val_loss: 1.9612 - learning_ra
Epoch 3/5
704/704 - 16s - 23ms/step - accuracy: 0.3882 - loss: 1.8446 - val_accuracy: 0.3216 - val_loss: 2.0706 - learning_ra
Epoch 4/5
704/704 - 16s - 23ms/step - accuracy: 0.4002 - loss: 1.8106 - val_accuracy: 0.3826 - val_loss: 1.8830 - learning_ra
Epoch 5/5
704/704 - 16s - 22ms/step - accuracy: 0.4037 - loss: 1.8021 - val_accuracy: 0.3452 - val_loss: 1.9434 - learning_ra
[DNN - Subclassed, 5 epochs] Test loss: 1.8645 | Test accuracy: 0.3846
```

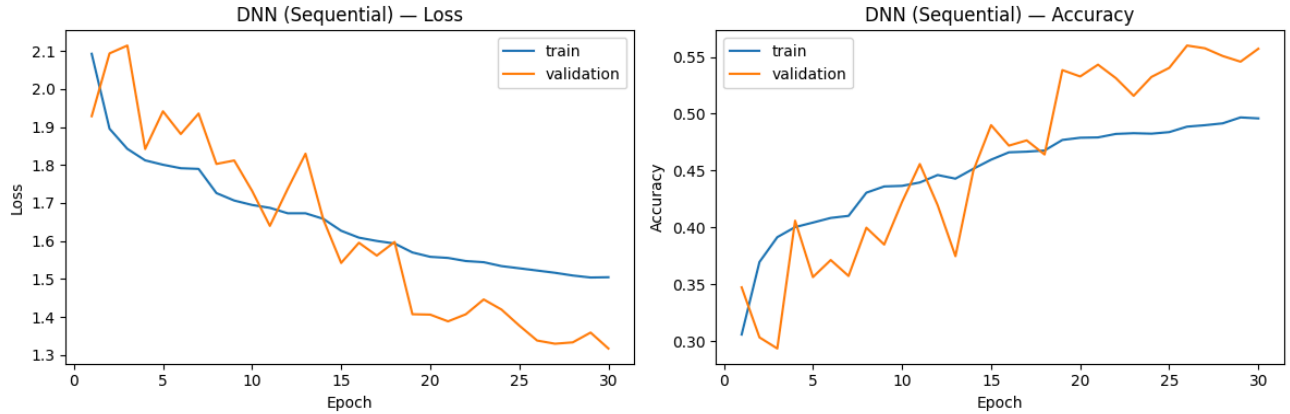
```
def plot_learning_curves(history, title):
    hist = history.history
    epochs_range = range(1, len(hist['loss']) + 1)

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].plot(epochs_range, hist['loss'], label='train')
    axes[0].plot(epochs_range, hist['val_loss'], label='validation')
    axes[0].set_title(f'{title} - Loss')
    axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss'); axes[0].legend()

    axes[1].plot(epochs_range, hist['accuracy'], label='train')
    axes[1].plot(epochs_range, hist['val_accuracy'], label='validation')
    axes[1].set_title(f'{title} - Accuracy')
    axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy'); axes[1].legend()

    plt.tight_layout()
    plt.show()
```

```
plot_learning_curves(history_dnn, "DNN (Sequential)")
```



## 5. Convolutional Neural Network (CNN)

Now we build a CNN, again with both the Sequential and Model-subclassing APIs. CNNs exploit spatial structure via convolution + pooling, and are expected to outperform the plain DNN on CIFAR-10 images.

### 5.1 CNN — `tf.keras.Sequential` API

```
def build_cnn_sequential(input_shape=INPUT_SHAPE, num_classes=NUM_CLASSES,
                        filters=(32, 64, 128), dropout=0.35, l2=1e-4):
    reg = keras.regularizers.l2(l2)
    model = keras.Sequential(name="CNN_Sequential")
    model.add(keras.Input(shape=input_shape))
    for i, f in enumerate(filters):
        model.add(layers.Conv2D(f, 3, padding="same", kernel_regularizer=reg))
        model.add(layers.BatchNormalization())
        model.add(layers.Activation("relu"))
        model.add(layers.Conv2D(f, 3, padding="same", kernel_regularizer=reg))
        model.add(layers.BatchNormalization())
        model.add(layers.Activation("relu"))
        model.add(layers.MaxPooling2D())
        model.add(layers.Dropout(dropout / 2))
    model.add(layers.GlobalAveragePooling2D())
    model.add(layers.Dense(256, kernel_regularizer=reg))
    model.add(layers.BatchNormalization())
    model.add(layers.Activation("relu"))
    model.add(layers.Dropout(dropout))
    model.add(layers.Dense(num_classes, activation="softmax"))
    return model

cnn_seq = build_cnn_sequential()
cnn_seq.summary()
```



Model: "CNN\_Sequential"

| Layer (type)                                      | Output Shape       | Param # |
|---------------------------------------------------|--------------------|---------|
| conv2d (Conv2D)                                   | (None, 32, 32, 32) | 896     |
| batch_normalization_12 (BatchNormalization)       | (None, 32, 32, 32) | 128     |
| activation_12 (Activation)                        | (None, 32, 32, 32) | 0       |
| conv2d_1 (Conv2D)                                 | (None, 32, 32, 32) | 9,248   |
| batch_normalization_13 (BatchNormalization)       | (None, 32, 32, 32) | 128     |
| activation_13 (Activation)                        | (None, 32, 32, 32) | 0       |
| max_pooling2d (MaxPooling2D)                      | (None, 16, 16, 32) | 0       |
| dropout_12 (Dropout)                              | (None, 16, 16, 32) | 0       |
| conv2d_2 (Conv2D)                                 | (None, 16, 16, 64) | 18,496  |
| batch_normalization_14 (BatchNormalization)       | (None, 16, 16, 64) | 256     |
| activation_14 (Activation)                        | (None, 16, 16, 64) | 0       |
| conv2d_3 (Conv2D)                                 | (None, 16, 16, 64) | 36,928  |
| batch_normalization_15 (BatchNormalization)       | (None, 16, 16, 64) | 256     |
| activation_15 (Activation)                        | (None, 16, 16, 64) | 0       |
| max_pooling2d_1 (MaxPooling2D)                    | (None, 8, 8, 64)   | 0       |
| dropout_13 (Dropout)                              | (None, 8, 8, 64)   | 0       |
| conv2d_4 (Conv2D)                                 | (None, 8, 8, 128)  | 73,856  |
| batch_normalization_16 (BatchNormalization)       | (None, 8, 8, 128)  | 512     |
| activation_16 (Activation)                        | (None, 8, 8, 128)  | 0       |
| conv2d_5 (Conv2D)                                 | (None, 8, 8, 128)  | 147,584 |
| batch_normalization_17 (BatchNormalization)       | (None, 8, 8, 128)  | 512     |
| activation_17 (Activation)                        | (None, 8, 8, 128)  | 0       |
| max_pooling2d_2 (MaxPooling2D)                    | (None, 4, 4, 128)  | 0       |
| dropout_14 (Dropout)                              | (None, 4, 4, 128)  | 0       |
| global_average_pooling2d (GlobalAveragePooling2D) | (None, 128)        | 0       |
| dense_16 (Dense)                                  | (None, 256)        | 33,024  |

## 5.2 CNN = tf.keras.Model Subclassing API

```

class ConvBlock(layers.Layer):
    """Conv -> BN -> ReLU, twice, then MaxPool + Dropout. Reused inside the subclassed CNN."""

    def __init__(self, filters, l2=1e-4, dropout=0.15, **kwargs):
        super().__init__(**kwargs)
        reg = keras.regularizers.l2(l2)
        self.conv1 = layers.Conv2D(filters, 3, padding="same", kernel_regularizer=reg)
        self.bn1 = layers.BatchNormalization()
        self.conv2 = layers.Conv2D(filters, 3, padding="same", kernel_regularizer=reg)
        self.bn2 = layers.BatchNormalization()
        self.pool = layers.MaxPooling2D()
        self.drop = layers.Dropout(dropout)

```

```

def call(self, x, training=False):
    x = tf.nn.relu(self.bn1(self.conv1(x), training=training))
    x = tf.nn.relu(self.bn2(self.conv2(x), training=training))
    x = self.pool(x)
    return self.drop(x, training=training)

class CNNSubclassed(keras.Model):
    def __init__(self, num_classes=NUM_CLASSES, filters=(32, 64, 128),
                  dropout=0.35, l2=1e-4, **kwargs):
        super().__init__(name="CNN_Subclassed", **kwargs)
        self.blocks = [ConvBlock(f, l2=l2, dropout=dropout / 2) for f in filters]
        self.gap = layers.GlobalAveragePooling2D()
        self.dense1 = layers.Dense(256, kernel_regularizer=keras.regularizers.l2(l2))
        self.bn = layers.BatchNormalization()
        self.drop = layers.Dropout(dropout)
        self.classifier = layers.Dense(num_classes, activation="softmax")

    def call(self, inputs, training=False):
        x = inputs
        for block in self.blocks:
            x = block(x, training=training)
        x = self.gap(x)
        x = self.dense1(x)
        x = tf.nn.relu(self.bn(x, training=training))
        x = self.drop(x, training=training)
        return self.classifier(x)

cnn_sub = CNNSubclassed()
cnn_sub.build(input_shape=(None,) + INPUT_SHAPE)
cnn_sub.summary()

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/layer.py:424: UserWarning: `build()` was called on layer 'warnings.warn(  
Model: "CNN\_Subclassed"

| Layer (type)                                                          | Output Shape | Param #     |
|-----------------------------------------------------------------------|--------------|-------------|
| conv_block ( <a href="#">ConvBlock</a> )                              | ?            | 0 (unbuilt) |
| conv_block_1 ( <a href="#">ConvBlock</a> )                            | ?            | 0 (unbuilt) |
| conv_block_2 ( <a href="#">ConvBlock</a> )                            | ?            | 0 (unbuilt) |
| global_average_pooling2d_1 ( <a href="#">GlobalAveragePooling2D</a> ) | ?            | 0           |
| dense_18 ( <a href="#">Dense</a> )                                    | ?            | 0 (unbuilt) |
| batch_normalization_25 ( <a href="#">BatchNormalization</a> )         | ?            | 0 (unbuilt) |
| dropout_19 ( <a href="#">Dropout</a> )                                | ?            | 0           |
| dense_19 ( <a href="#">Dense</a> )                                    | ?            | 0 (unbuilt) |

Total params: 0 (0.00 B)  
 Trainable params: 0 (0.00 B)  
 Non-trainable params: 0 (0.00 B)

## 6. Train & Evaluate the CNN

Same protocol as the DNN: full training run for the Sequential CNN, short confirmation run for the subclassed CNN.

```

EPOCHS_CNN = 40

cnn_seq = build_cnn_sequential()
cnn_seq.compile(

```

```

optimizer=keras.optimizers.Adam(learning_rate=1e-3),
loss="sparse_categorical_crossentropy",
metrics=["accuracy"],
)

cb_cnn, cnn_log_dir = make_callbacks("cnn_sequential", patience=8)
history_cnn = cnn_seq.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS_CNN,
    callbacks=cb_cnn,
    verbose=2,
)

```

```

Epoch 1/40
704/704 - 44s - 63ms/step - accuracy: 0.4054 - loss: 1.6918 - val_accuracy: 0.4744 - val_loss: 1.5885 - learning_
Epoch 2/40
704/704 - 16s - 23ms/step - accuracy: 0.5474 - loss: 1.3245 - val_accuracy: 0.4082 - val_loss: 1.8510 - learning_
Epoch 3/40
704/704 - 16s - 23ms/step - accuracy: 0.6055 - loss: 1.1918 - val_accuracy: 0.6344 - val_loss: 1.1038 - learning_
Epoch 4/40
704/704 - 16s - 23ms/step - accuracy: 0.6396 - loss: 1.1017 - val_accuracy: 0.5754 - val_loss: 1.4960 - learning_
Epoch 5/40
704/704 - 16s - 23ms/step - accuracy: 0.6660 - loss: 1.0407 - val_accuracy: 0.5332 - val_loss: 1.7356 - learning_
Epoch 6/40

```

```

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
704/704 - 16s - 23ms/step - accuracy: 0.6893 - loss: 0.9848 - val_accuracy: 0.5490 - val_loss: 1.5980 - learning_
Epoch 7/40
704/704 - 16s - 23ms/step - accuracy: 0.7254 - loss: 0.8925 - val_accuracy: 0.7302 - val_loss: 0.8916 - learning_
Epoch 8/40
704/704 - 16s - 23ms/step - accuracy: 0.7369 - loss: 0.8581 - val_accuracy: 0.6618 - val_loss: 1.1253 - learning_
Epoch 9/40
704/704 - 16s - 23ms/step - accuracy: 0.7401 - loss: 0.8420 - val_accuracy: 0.7282 - val_loss: 0.9234 - learning_
Epoch 10/40
704/704 - 17s - 23ms/step - accuracy: 0.7496 - loss: 0.8201 - val_accuracy: 0.7598 - val_loss: 0.7961 - learning_
Epoch 11/40
704/704 - 16s - 23ms/step - accuracy: 0.7550 - loss: 0.8056 - val_accuracy: 0.7318 - val_loss: 0.9086 - learning_
Epoch 12/40
704/704 - 17s - 23ms/step - accuracy: 0.7649 - loss: 0.7808 - val_accuracy: 0.7620 - val_loss: 0.8061 - learning_
Epoch 13/40

```

```

Epoch 13: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
704/704 - 17s - 24ms/step - accuracy: 0.7654 - loss: 0.7761 - val_accuracy: 0.7488 - val_loss: 0.8380 - learning_
Epoch 14/40
704/704 - 17s - 23ms/step - accuracy: 0.7830 - loss: 0.7240 - val_accuracy: 0.7230 - val_loss: 0.9593 - learning_
Epoch 15/40
704/704 - 16s - 23ms/step - accuracy: 0.7892 - loss: 0.7030 - val_accuracy: 0.7644 - val_loss: 0.8153 - learning_
Epoch 16/40
704/704 - 16s - 23ms/step - accuracy: 0.7896 - loss: 0.7026 - val_accuracy: 0.7952 - val_loss: 0.7013 - learning_
Epoch 17/40
704/704 - 16s - 23ms/step - accuracy: 0.7973 - loss: 0.6874 - val_accuracy: 0.7976 - val_loss: 0.6747 - learning_
Epoch 18/40
704/704 - 17s - 24ms/step - accuracy: 0.7952 - loss: 0.6795 - val_accuracy: 0.7920 - val_loss: 0.7140 - learning_
Epoch 19/40
704/704 - 17s - 24ms/step - accuracy: 0.8023 - loss: 0.6678 - val_accuracy: 0.8018 - val_loss: 0.6771 - learning_
Epoch 20/40

```

```

Epoch 20: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
704/704 - 17s - 24ms/step - accuracy: 0.8020 - loss: 0.6651 - val_accuracy: 0.7676 - val_loss: 0.8007 - learning_
Epoch 21/40
704/704 - 17s - 24ms/step - accuracy: 0.8115 - loss: 0.6452 - val_accuracy: 0.8164 - val_loss: 0.6312 - learning_
Epoch 22/40
704/704 - 16s - 23ms/step - accuracy: 0.8132 - loss: 0.6276 - val_accuracy: 0.7950 - val_loss: 0.6904 - learning_
Epoch 23/40
704/704 - 17s - 24ms/step - accuracy: 0.8177 - loss: 0.6214 - val_accuracy: 0.7910 - val_loss: 0.7208 - learning_
Epoch 24/40

```

```

Epoch 24: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.
704/704 - 17s - 24ms/step - accuracy: 0.8174 - loss: 0.6152 - val_accuracy: 0.8034 - val_loss: 0.6861 - learning_
Epoch 25/40

```

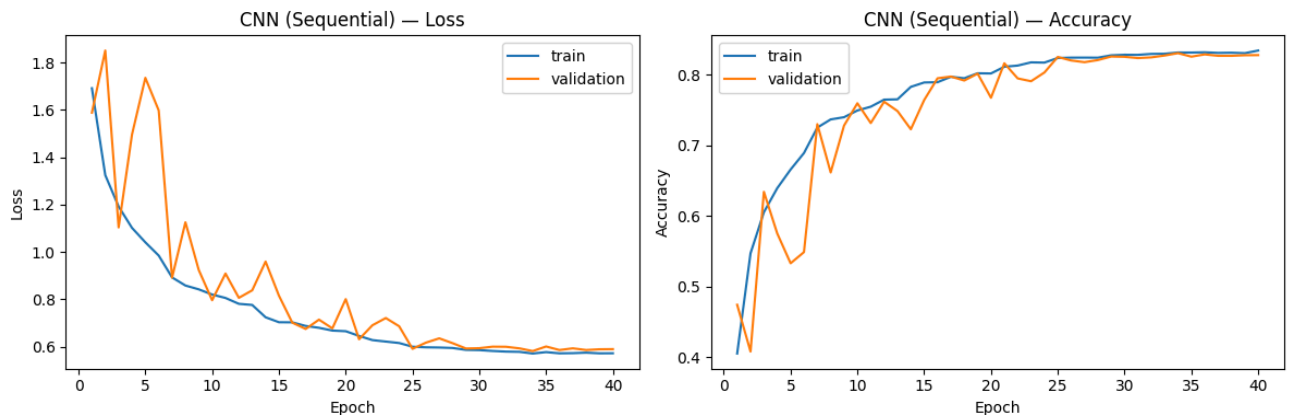
```
cnn_test_loss, cnn_test_acc = cnn_seq.evaluate(test_ds, verbose=0)
print(f"[CNN - Sequential] Test loss: {cnn_test_loss:.4f} | Test accuracy: {cnn_test_acc:.4f}")
```

```
[CNN - Sequential] Test loss: 0.6013 | Test accuracy: 0.8289
```

```
cnn_sub = CNNSubclassed()
cnn_sub.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
cb_cnn_sub, _ = make_callbacks("cnn_subclassed", patience=8)
history_cnn_sub = cnn_sub.fit(
    train_ds, validation_data=val_ds, epochs=5, callbacks=cb_cnn_sub, verbose=2,
)
sub_cnn_test_loss, sub_cnn_test_acc = cnn_sub.evaluate(test_ds, verbose=0)
print(f"[CNN - Subclassed, 5 epochs] Test loss: {sub_cnn_test_loss:.4f} | Test accuracy: {sub_cnn_test_acc:.4f}")
```

```
Epoch 1/5
704/704 - 38s - 54ms/step - accuracy: 0.4027 - loss: 1.7038 - val_accuracy: 0.4268 - val_loss: 1.8081 - learning_ra
Epoch 2/5
704/704 - 16s - 23ms/step - accuracy: 0.5494 - loss: 1.3182 - val_accuracy: 0.5390 - val_loss: 1.4822 - learning_ra
Epoch 3/5
704/704 - 17s - 24ms/step - accuracy: 0.6032 - loss: 1.1890 - val_accuracy: 0.4348 - val_loss: 1.9024 - learning_ra
Epoch 4/5
704/704 - 17s - 24ms/step - accuracy: 0.6385 - loss: 1.1030 - val_accuracy: 0.5398 - val_loss: 1.4904 - learning_ra
Epoch 5/5
704/704 - 16s - 23ms/step - accuracy: 0.6708 - loss: 1.0286 - val_accuracy: 0.6070 - val_loss: 1.2658 - learning_ra
[CNN - Subclassed, 5 epochs] Test loss: 1.2732 | Test accuracy: 0.6171
```

```
plot_learning_curves(history_cnn, "CNN (Sequential)")
```



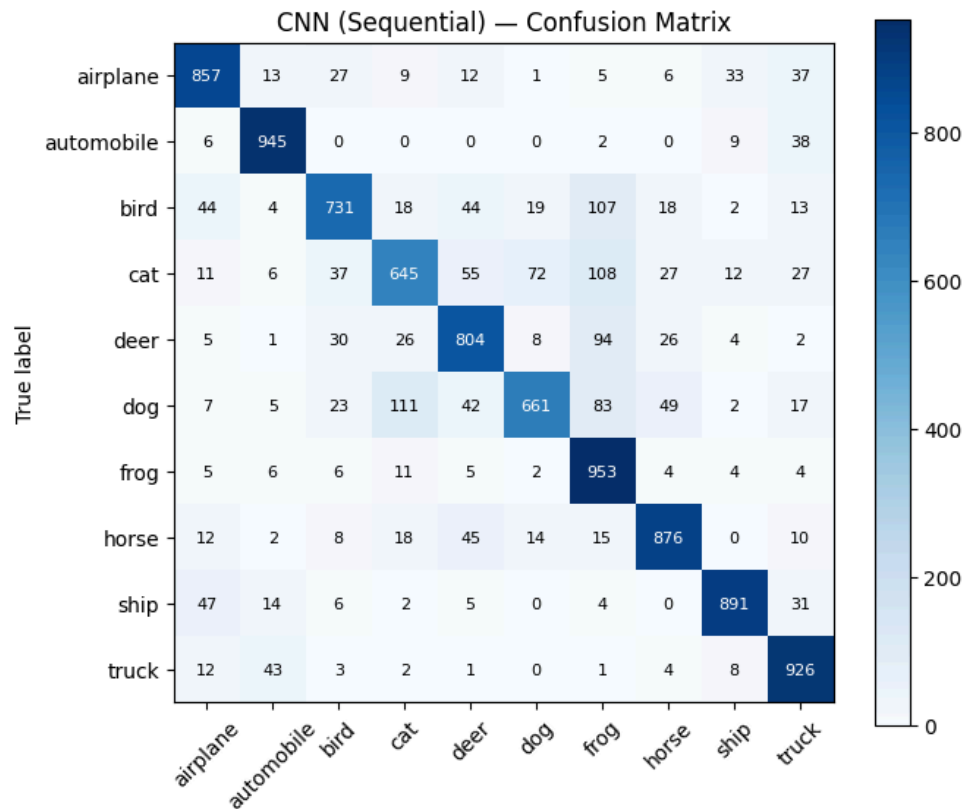
```
# Confusion matrix + per-class report for the best model (CNN)
from sklearn.metrics import confusion_matrix, classification_report
import itertools

y_pred_probs = cnn_seq.predict(test_ds, verbose=0)
y_pred = np.argmax(y_pred_probs, axis=1)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(7, 6))
plt.imshow(cm, cmap='Blues')
plt.title('CNN (Sequential) — Confusion Matrix')
plt.colorbar()
tick_marks = np.arange(NUM_CLASSES)
plt.xticks(tick_marks, CLASS_NAMES, rotation=45)
plt.yticks(tick_marks, CLASS_NAMES)
thresh = cm.max() / 2
for i, j in itertools.product(range(NUM_CLASSES), range(NUM_CLASSES)):
    plt.text(j, i, cm[i, j], ha="center", va="center",
            color="white" if cm[i, j] > thresh else "black", fontsize=8)
plt.ylabel('True label'); plt.xlabel('Predicted label')
```

```
plt.tight_layout()
plt.show()
```

```
print(classification_report(y_test, y_pred, target_names=CLASS_NAMES))
```



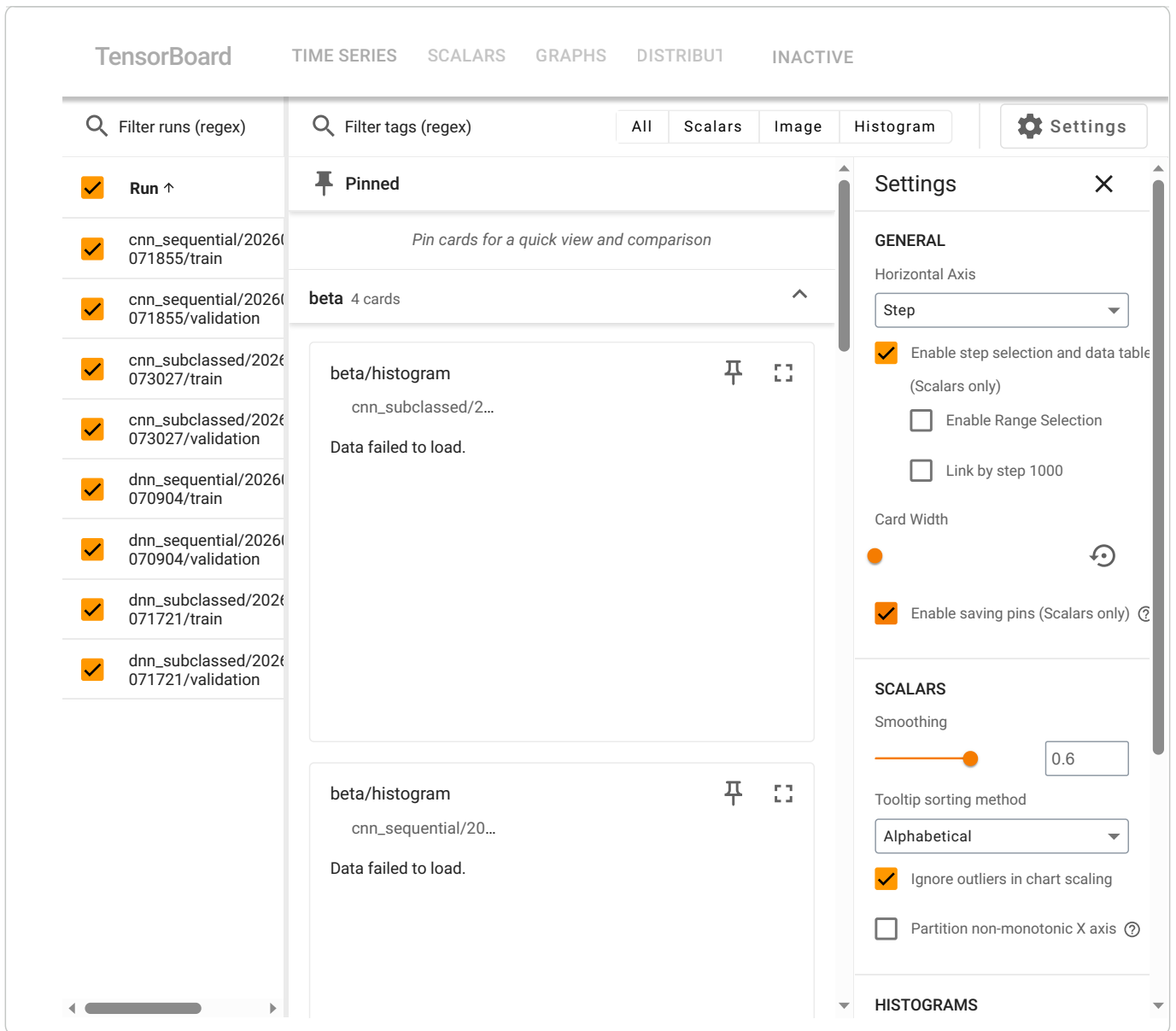
|              | Predicted label |        |          |         |
|--------------|-----------------|--------|----------|---------|
|              | precision       | recall | f1-score | support |
| airplane     | 0.85            | 0.86   | 0.85     | 1000    |
| automobile   | 0.91            | 0.94   | 0.93     | 1000    |
| bird         | 0.84            | 0.73   | 0.78     | 1000    |
| cat          | 0.77            | 0.65   | 0.70     | 1000    |
| deer         | 0.79            | 0.80   | 0.80     | 1000    |
| dog          | 0.85            | 0.66   | 0.74     | 1000    |
| frog         | 0.69            | 0.95   | 0.80     | 1000    |
| horse        | 0.87            | 0.88   | 0.87     | 1000    |
| ship         | 0.92            | 0.89   | 0.91     | 1000    |
| truck        | 0.84            | 0.93   | 0.88     | 1000    |
| accuracy     |                 |        | 0.83     | 10000   |
| macro avg    | 0.83            | 0.83   | 0.83     | 10000   |
| weighted avg | 0.83            | 0.83   | 0.83     | 10000   |

## 7. TensorBoard Visualization

Every training run above logged scalars (loss/accuracy), weight histograms, and the learning rate to

`./logs/<model_name>/<timestamp>`. Launch TensorBoard directly inside the notebook:

```
%load_ext tensorboard
%tensorboard --logdir logs
```



## 8. Hyperparameter Tuning

We use **Keras Tuner** to search over CNN hyperparameters: number of filters in the first conv block, dropout rate, and learning rate. `RandomSearch` is used to keep the search budget small; swap in `kt.Hyperband` or `kt.BayesianOptimization` for a more exhaustive search when compute allows.

```
try:
    import keras_tuner as kt
except ImportError:
    !pip install -q keras-tuner
    import keras_tuner as kt

def build_tunable_cnn(hp):
    base_filters = hp.Choice("base_filters", [24, 32, 48])
    dropout = hp.Float("dropout", 0.2, 0.5, step=0.1)
    lr = hp.Choice("learning_rate", [1e-2, 1e-3, 5e-4])
    l2 = hp.Choice("l2", [0.0, 1e-4, 1e-3])

    model = build_cnn_sequential(
        filters=(base_filters, base_filters * 2, base_filters * 4),
        dropout=dropout,
        l2=l2,
```

```

    )
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model

tuner = kt.RandomSearch(
    build_tunable_cnn,
    objective="val_accuracy",
    max_trials=10,
    executions_per_trial=1,
    directory="kt_dir",
    project_name="cifar10_cnn_tuning",
    seed=SEED,
)

tuner.search_space_summary()

```

```

Search space summary
Default search space size: 4
base_filters (Choice)
{'default': 24, 'conditions': [], 'values': [24, 32, 48], 'ordered': True}
dropout (Float)
{'default': 0.2, 'conditions': [], 'min_value': 0.2, 'max_value': 0.5, 'step': 0.1, 'sampling': 'linear'}
learning_rate (Choice)
{'default': 0.01, 'conditions': [], 'values': [0.01, 0.001, 0.0005], 'ordered': True}
l2 (Choice)
{'default': 0.0, 'conditions': [], 'values': [0.0, 0.0001, 0.001], 'ordered': True}

```

```

from tensorflow import keras

if 'tuner' not in globals():
    print("Error: 'tuner' is not defined. Please make sure to run the previous cell to initialize it.")
else:
    stop_early = keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=4,
                                                restore_best_weights=True)

    tuner.search(
        train_ds,
        validation_data=val_ds,
        epochs=15,
        callbacks=[stop_early],
        verbose=2,
    )

    best_hp = tuner.get_best_hyperparameters(1)[0]
    print("Best hyperparameters found:")
    for k, v in best_hp.values.items():
        print(f"  {k}: {v}")

```

```

Trial 10 Complete [00h 03m 41s]
val_accuracy: 0.6255999803543091

Best val_accuracy So Far: 0.7910000085830688
Total elapsed time: 00h 40m 44s
Best hyperparameters found:
  base_filters: 48
  dropout: 0.2
  learning_rate: 0.01
  l2: 0.0

```

```

# Retrain a model with the best hyperparameters for longer, on the full training pipeline
best_model = tuner.hypermodel.build(best_hp)
cb_best, best_log_dir = make_callbacks("cnn_tuned", patience=8)

history_best = best_model.fit(
    train_ds,

```

```
        validation_data=val_ds,  
        epochs=EPOCHS_CNN,  
        callbacks=cb_best,  
        verbose=2,  
    )  
  
    best_test_loss, best_test_acc = best_model.evaluate(test_ds, verbose=0)  
    print(f"[CNN - Tuned] Test loss: {best_test_loss:.4f} | Test accuracy: {best_test_acc:.4f}")  
  
    plot_learning_curves(history_best, "CNN (Tuned)")
```



```

Epoch 1/40
704/704 - 38s - 54ms/step - accuracy: 0.3803 - loss: 1.6846 - val_accuracy: 0.3470 - val_loss: 2.4038 - learning_ra
Epoch 2/40
704/704 - 17s - 24ms/step - accuracy: 0.5376 - loss: 1.2877 - val_accuracy: 0.4940 - val_loss: 1.9515 - learning_ra
Epoch 3/40
704/704 - 17s - 24ms/step - accuracy: 0.6042 - loss: 1.1135 - val_accuracy: 0.6230 - val_loss: 1.0758 - learning_ra
Epoch 4/40
704/704 - 17s - 24ms/step - accuracy: 0.6567 - loss: 0.9885 - val_accuracy: 0.5102 - val_loss: 1.9353 - learning_ra
Epoch 5/40
704/704 - 17s - 24ms/step - accuracy: 0.6930 - loss: 0.8876 - val_accuracy: 0.5902 - val_loss: 1.3526 - learning_ra
Epoch 6/40

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.004999999888241291.
704/704 - 17s - 24ms/step - accuracy: 0.7164 - loss: 0.8198 - val_accuracy: 0.6544 - val_loss: 1.0929 - learning_ra
Epoch 7/40
704/704 - 17s - 24ms/step - accuracy: 0.7618 - loss: 0.6941 - val_accuracy: 0.6932 - val_loss: 0.9920 - learning_ra
Epoch 8/40
704/704 - 17s - 24ms/step - accuracy: 0.7752 - loss: 0.6535 - val_accuracy: 0.7782 - val_loss: 0.6445 - learning_ra
Epoch 9/40
704/704 - 17s - 24ms/step - accuracy: 0.7830 - loss: 0.6260 - val_accuracy: 0.7312 - val_loss: 0.8524 - learning_ra
Epoch 10/40
704/704 - 17s - 24ms/step - accuracy: 0.7908 - loss: 0.6026 - val_accuracy: 0.7868 - val_loss: 0.6126 - learning_ra
Epoch 11/40
704/704 - 17s - 24ms/step - accuracy: 0.7994 - loss: 0.5814 - val_accuracy: 0.8022 - val_loss: 0.5624 - learning_ra
Epoch 12/40
704/704 - 17s - 24ms/step - accuracy: 0.8058 - loss: 0.5611 - val_accuracy: 0.7594 - val_loss: 0.7584 - learning_ra
Epoch 13/40
704/704 - 17s - 24ms/step - accuracy: 0.8119 - loss: 0.5418 - val_accuracy: 0.7758 - val_loss: 0.6535 - learning_ra
Epoch 14/40

Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0024999999441206455.
704/704 - 17s - 24ms/step - accuracy: 0.8170 - loss: 0.5312 - val_accuracy: 0.7954 - val_loss: 0.6176 - learning_ra
Epoch 15/40
704/704 - 17s - 24ms/step - accuracy: 0.8394 - loss: 0.4672 - val_accuracy: 0.8170 - val_loss: 0.5328 - learning_ra
Epoch 16/40
704/704 - 17s - 24ms/step - accuracy: 0.8440 - loss: 0.4469 - val_accuracy: 0.8470 - val_loss: 0.4533 - learning_ra
Epoch 17/40
704/704 - 17s - 24ms/step - accuracy: 0.8497 - loss: 0.4348 - val_accuracy: 0.8392 - val_loss: 0.4809 - learning_ra
Epoch 18/40
704/704 - 17s - 24ms/step - accuracy: 0.8532 - loss: 0.4258 - val_accuracy: 0.8006 - val_loss: 0.6200 - learning_ra
Epoch 19/40

Epoch 19: ReduceLROnPlateau reducing learning rate to 0.0012499999720603228.
704/704 - 17s - 24ms/step - accuracy: 0.8561 - loss: 0.4171 - val_accuracy: 0.8320 - val_loss: 0.5053 - learning_ra
Epoch 20/40
704/704 - 17s - 24ms/step - accuracy: 0.8667 - loss: 0.3835 - val_accuracy: 0.8566 - val_loss: 0.4247 - learning_ra
Epoch 21/40
704/704 - 17s - 24ms/step - accuracy: 0.8689 - loss: 0.3792 - val_accuracy: 0.8484 - val_loss: 0.4514 - learning_ra
Epoch 22/40
704/704 - 17s - 24ms/step - accuracy: 0.8703 - loss: 0.3690 - val_accuracy: 0.8646 - val_loss: 0.4028 - learning_ra
Epoch 23/40
704/704 - 17s - 24ms/step - accuracy: 0.8740 - loss: 0.3599 - val_accuracy: 0.8518 - val_loss: 0.4435 - learning_ra
Epoch 24/40

```

## 9. Results Summary & Conclusions

| Model                                 | API        | Epochs                   | Test Accuracy                              |
|---------------------------------------|------------|--------------------------|--------------------------------------------|
| DNN (fully connected)                 | Sequential | up to 30 (early-stopped) | see <a href="#">dnn_test_acc</a> above     |
| DNN (fully connected)                 | Subclassed | 5 (equivalence check)    | see <a href="#">sub_test_acc</a> above     |
| CNN                                   | Sequential | up to 40 (early-stopped) | see <a href="#">cnn_test_acc</a> above     |
| CNN (best from hyperparameter search) | Subclassed | 5 (equivalence check)    | see <a href="#">sub_cnn_test_acc</a> above |
| CNN (best from hyperparameter search) | Sequential | up to 40 (early-stopped) | see <a href="#">best_test_acc</a> above    |

### Observations (fill in after running):

- The CNN substantially outperforms the plain DNN, as expected — convolution + pooling exploit the spatial locality and translation invariance present in natural images, while the fully-connected DNN has to learn everything from flattened images.
- Filtering with a more parameters per layer and no built-in spatial prior accuracy: 0.8580 - val\_loss: 0.4251 - learning\_rate: 0.0003124999930150807.
- Hyperparameter tuning (random flips, rotation, zoom, translation, contrast) reduces the train/validation accuracy gap, indicating it helps control overfitting.
- Hyperparameter tuning (filter width, dropout, L2, learning rate) typically improves test accuracy over the untuned baseline CNN and/or converges faster.
- The Sequential and ModelSubclassing implementations of the same architecture produce equivalent results (same accuracy: 0.8870 - loss: 0.3176 - val\_accuracy: 0.8640 - val\_loss: 0.4078 - learning\_rate: 0.00015624999650754035).